# Multiheaded attention and its gradient

Siavash Ahmadi

The idea of "attention" in the transformer is actually really simple. Below, I'll be repeating the same idea multiple times each time with slightly more bells and whistles to hopefully make this easier to follow.

## 1 Attention

The core of the Transformer architecture is the attention function, which is surprisingly simple and straightforward to describe. Attention is a function of three inputs arguments (which are matrices) and one output argument (which is a matrix), and can be separated into three semantically distinct steps:

1. In-projections
2. Scaled dot-product attention
3. Out-projection

Note that these three steps are just names we're using to partition the operations of the attention function in an easier to digest way. Attention is just the composition of several function. These the names refer to the following equations:

1. In-projections

$$\mathbf{Q} = \mathbf{q}\mathbf{W}^{\mathbf{q}}, \quad \mathbf{K} = \mathbf{k}\mathbf{W}^{\mathbf{k}}, \quad \mathbf{V} = \mathbf{v}\mathbf{W}^{\mathbf{v}}$$

2. Scaled dot-product attention

$$\mathbf{A} = \text{softmax}(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}})\mathbf{V}$$

3. Out-projection

$$\mathbf{Z} = \mathbf{A}\mathbf{W}^{\mathbf{o}}$$

## 2 Multiheaded Attention

The transformer uses multiheaded attention which is just a fancy-sounding way to saying it repeats the above operations multiple times using different values for the four $\mathbf{W}$ parameter matrices.

The transformer multiheaded attention boils down to the following equations, where we now use the $i = 1, \ldots, h$ subscripts to distinguish between the different *heads* and write the whole thing a bit more formally:

$$\mathbf{Q}_i = \mathbf{q}\mathbf{W}_i^{\mathbf{q}}, \quad \mathbf{K}_i = \mathbf{k}\mathbf{K}_i^{\mathbf{k}}, \quad \mathbf{V}_i = \mathbf{v}\mathbf{W}_i^{\mathbf{v}}$$
$$\mathbf{A}_i = \text{softmax}(\frac{\mathbf{Q}_i\mathbf{K}_i^\top}{\sqrt{d_k}})\mathbf{V}_i$$
$$\mathbf{Z} = \left[\mathbf{A}_1\mathbf{A}_2\cdots\mathbf{A}_h\right]\mathbf{W}^{\mathbf{o}}$$

Putting it all together we get:

$$\mathbf{Z}(\mathbf{q},\mathbf{k},\mathbf{v}) = \begin{bmatrix} \text{softmax}(\frac{\mathbf{q}\mathbf{W}_1^{\mathbf{q}}(\mathbf{k}\mathbf{K}_1^{\mathbf{k}})^\top}{\sqrt{d_k}})\mathbf{v}\mathbf{W}_1^{\mathbf{v}} \\ \text{softmax}(\frac{\mathbf{q}\mathbf{W}_2^{\mathbf{q}}(\mathbf{k}\mathbf{K}_2^{\mathbf{k}})^\top}{\sqrt{d_k}})\mathbf{v}\mathbf{W}_2^{\mathbf{v}} \\ \vdots \\ \text{softmax}(\frac{\mathbf{q}\mathbf{W}_h^{\mathbf{q}}(\mathbf{k}\mathbf{K}_h^{\mathbf{k}})^\top}{\sqrt{d_k}})\mathbf{v}\mathbf{W}_h^{\mathbf{v}} \end{bmatrix}^\top \mathbf{W}^{\mathbf{o}} \tag{1}$$

where the three-input $\mathbf{Z}$ (the multiheaded attention with $h$ heads) is parameterized by $3h+1$ matrices: $\mathbf{W}_i{}^{\mathbf{q}}$, $\mathbf{W}_i{}^{\mathbf{k}}$, $\mathbf{W}_i{}^{\mathbf{v}}$, and $\mathbf{W}^{\mathbf{o}}$, $i = 1,\ldots,h$.

The "hardest" thing about implementing multiheaded attention from scratch is managing the matrix shapes effectively so as to make the matrix multiplications efficient.

# 3 Full description of multiheaded attention operations

## 3.1 In projections

| Inputs | Parameters | Forward pass operations | Outputs |
|---|---|---|---|
| $\mathbf{q} \in \mathbb{R}^{N\times L_q\times d_{\text{model}}}$ | $\mathbf{W_q} \in \mathbb{R}^{h\times d_{\text{model}}\times d_k}$ | `Q = np.dot(q,Wq)`<br>`Q = np.moveaxis(Q, -2, 0)` | $\mathbf{Q} \in \mathbb{R}^{h\times N\times L_q\times d_k}$ |
| $\mathbf{k} \in \mathbb{R}^{N\times L_k\times kdim}$ | $\mathbf{W_k} \in \mathbb{R}^{h\times kdim\times d_k}$ | `K = np.dot(k,Wk)`<br>`K = np.moveaxis(K, -2, 0)` | $\mathbf{K} \in \mathbb{R}^{h\times N\times L_k\times d_k}$ |
| $\mathbf{v} \in \mathbb{R}^{N\times L_k\times vdim}$ | $\mathbf{W_v} \in \mathbb{R}^{h\times vdim\times d_v}$ | `V = np.dot(v,Wv)`<br>`V = np.moveaxis(V, -2, 0)` | $\mathbf{V} \in \mathbb{R}^{h\times N\times L_k\times d_v}$ |

### 3.2 Scaled dot product attention

#### 3.2.1 Scaled dot product

| Inputs | Forward pass operations | Outputs |
|---|---|---|
| $\mathbf{Q} \in \mathbb{R}^{h \times N \times L_q \times d_k}$<br><br>$\mathbf{K} \in \mathbb{R}^{h \times N \times L_k \times d_k}$ | `s = 1 / np.sqrt(d_k)`<br>`S = Q @ np.swapaxes(K, -1, -2) * s` | $\mathbf{S} \in \mathbb{R}^{h \times N \times L_q \times L_k}$ |

#### 3.2.2 Masked Softmax

For the softmax gradient, see softmax and its gradient (PDF).

#### 3.2.3 Attention-weighted values

### 3.3 Out projection

## 4 The gradients (backward pass)